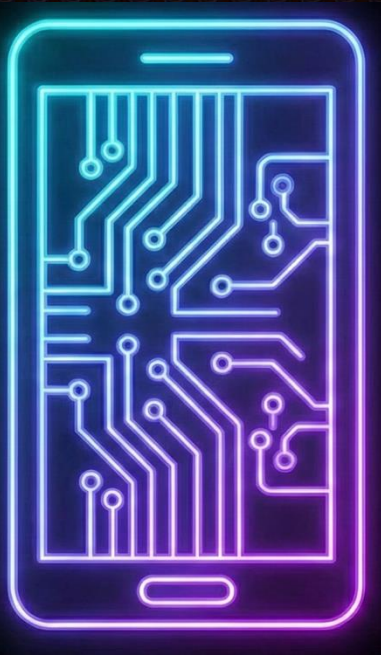


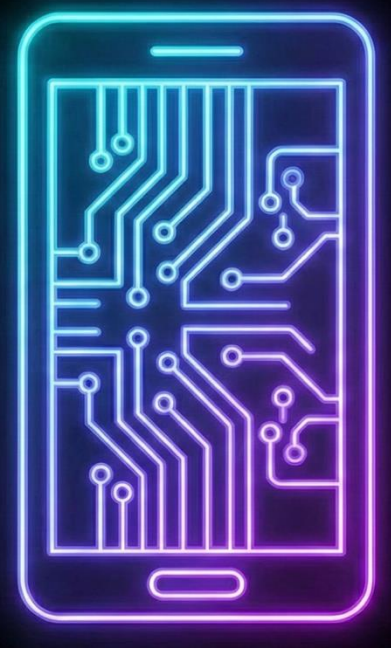


Beyond the API: Cracking the Shell of Mobile App Security

Tools and techniques to fit deep mobile app binary analysis into tight engagement windows without sacrificing coverage

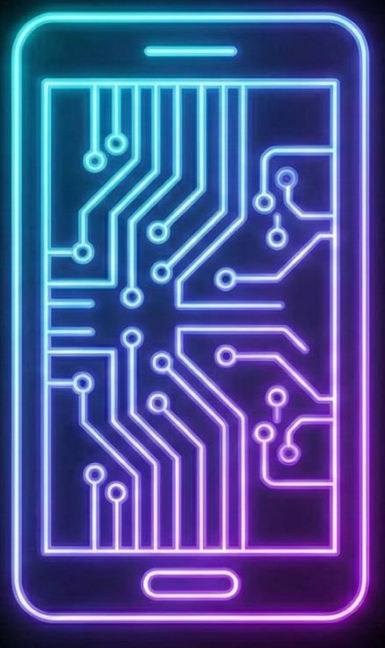
Steve Campbell

Agenda



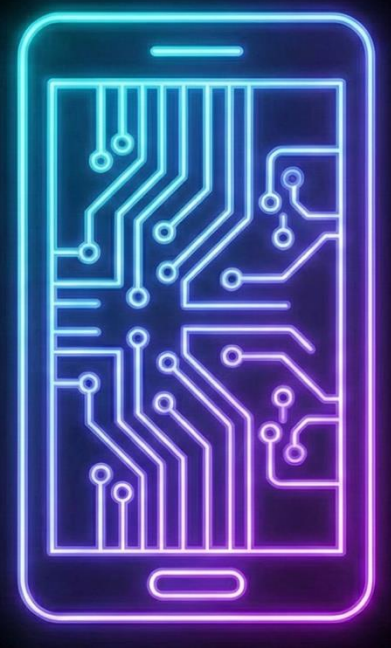
1. Introduction
2. Purpose
3. Challenges
4. Introduction to Android/iOS
5. Common tools
6. Tool Demos
7. Intro to OWASP MASTG
8. Example automation
9. Example findings
10. Tool drop
11. Questions and Comments

Whoami



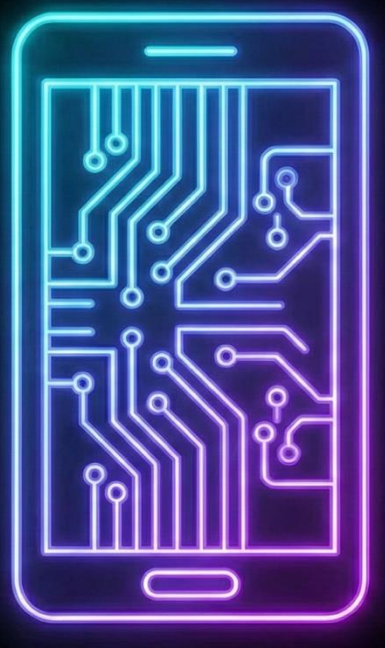
- Retired Navy veteran @cylentsec www.cylentsec.com
- Offensive security consultant with over 21 years experience
- Technical Lead CDW Offensive Security Team
- Published CVE: 3 Critical Risk, 2 High, 3 Medium
- Published Author: "Bash Shell Scripting for Pentesters"
- Contributor to Metasploit and other open-source projects

Your first mobile app pentest assignment:



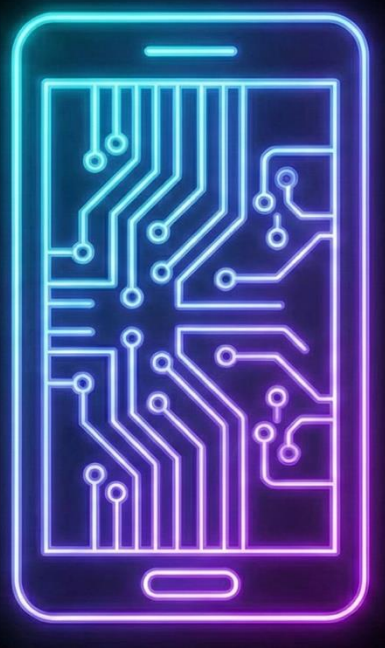
- What, no shells? Boring! (Bugs are typically buried deeper than your skill)
- WTH am I doing here?
- The mobile app is frequently treated as an API request generator
- Typical: Run MobSF scan => Bypass root/jailbreak detection and cert pinning => Test the API in Burp where you're comfortable

Why should you care?



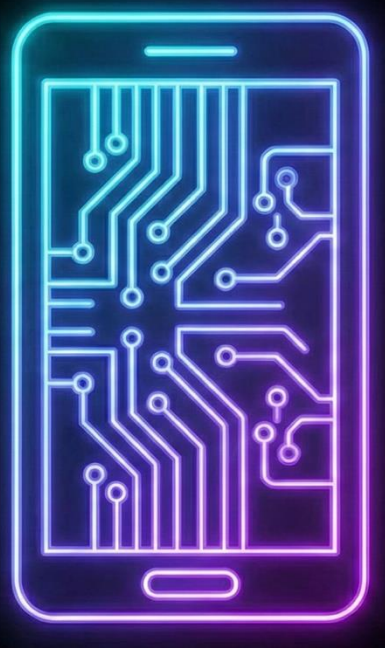
- Less competition: Bug bounty \$ and career development
- Critical vulnerabilities lurk much deeper than automated tools can reach
- Learned skills are transferrable to other architectures
- It's fun, interesting, and profitable once you learn how to uncover deeply hidden bugs

Challenges



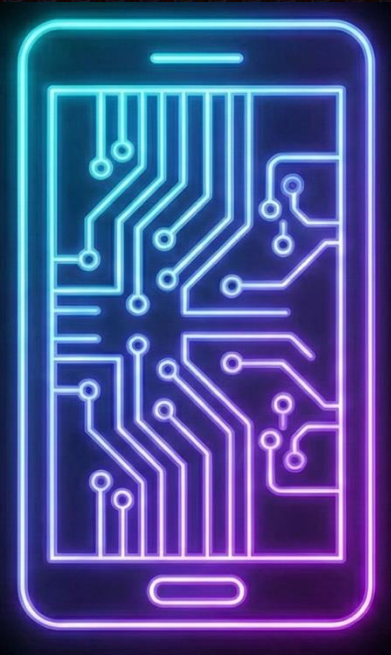
- Managing a test device and tool dependencies can be frustrating
- The mobile app sandbox is locked down tight
- Available training teaches bug class exploitation, not how to approach a real-world mobile app security assessment
- Mobile app pentest compressed schedule
- Hacking iOS apps are much more difficult compared to Android

Intro to Android



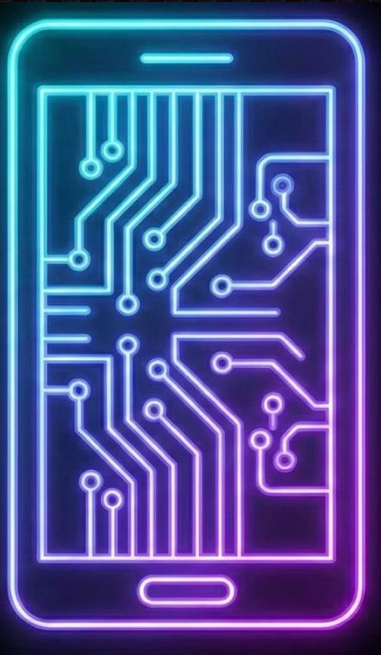
- The Execution Environment: Android apps (Java/Kotlin) compile to Dalvik Executable (DEX) bytecode, executed by the Android Runtime (ART)
- The System Architecture: The platform is a layered stack—spanning from the Linux Kernel and Hardware Abstraction Layer (HAL) up to the Framework APIs
- Application Sandboxing Model: The OS enforces strict isolation; each application runs within its own sandbox, assigned a unique Linux User ID (UID), and is restricted by a manifest-declared permission model.
- Open-source; Easy decompilation to Java

Managing the mobile device: Android



- Android Studio virtual device
 - Easy/Open
 - Free
- One VM for access to Google Play Store, one rooted device for testing

Android Studio Device Manager



Device Manager

☰ +

^ Name

- Medium Phone
Android 15.0 ("VanillaIceCream") | arm64
- Pixel 5 rooted
Android 11.0 ("R") | arm64

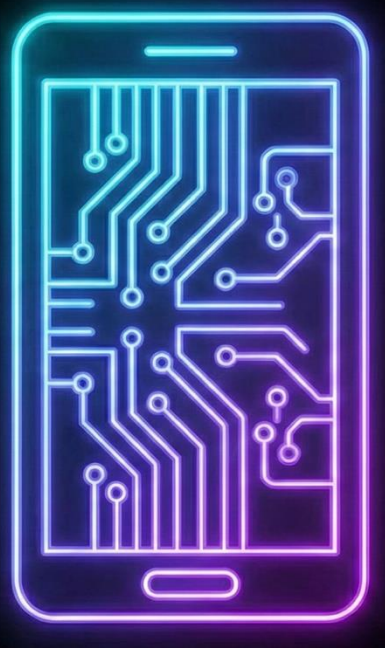
9:46

Sunday, Mar 8 ☁ 71°F

API	Type	
35	Virtual	▶ ⋮
30	Virtual	◻ ⋮

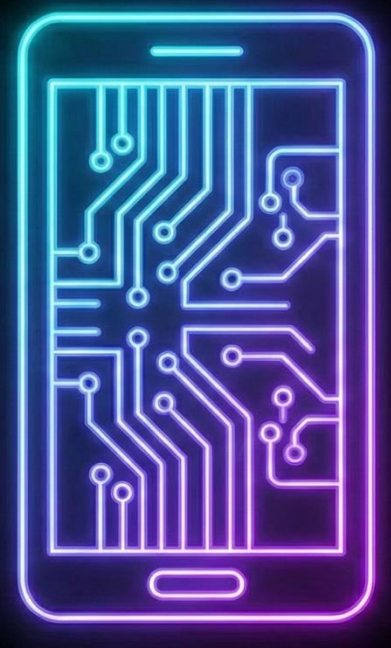
⏻ 🔊 🔊 📷 🔍 📄 🔄 ⏪ ⏩ ⏹ ⋮

Intro to iOS



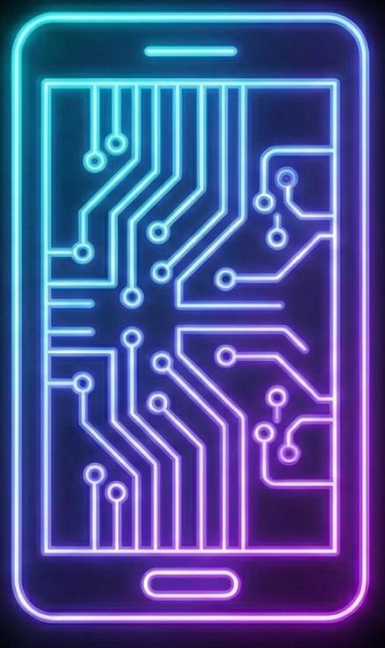
- The Execution Environment: iOS apps (Swift/Objective-C) compile directly to native ARM64 machine code packaged as Mach-O binaries, executed directly by the processor without a runtime virtual machine.
- The System Architecture: The platform is a layered stack based on Darwin—spanning from the XNU Kernel and Core OS layer, up through Core Services and the Cocoa Touch framework.
- Application Sandboxing Model: The OS enforces strict isolation via the App Sandbox (Seatbelt); third-party apps run under a shared user (mobile) but are sandboxed by Mandatory Access Control (MAC) policies and restricted by cryptographically signed Entitlements.
- Closed-source; Stripped native binaries

Managing the mobile device: iOS



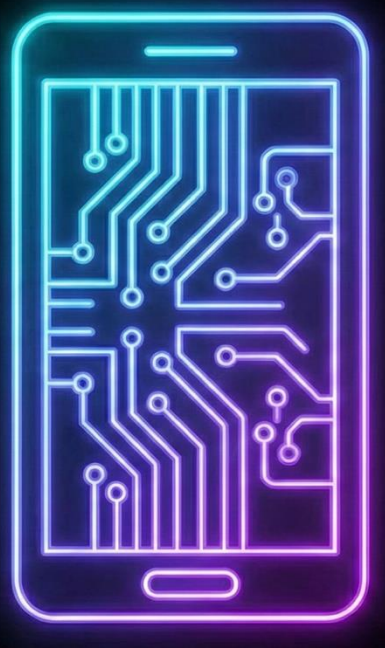
- No free virtual device software
- Corellium virtual devices cost \$3 per hour; are useful for paid gigs
- Jailbreaks are much more complex than rooting an Android
- Jailbreaks are limited on newer iOS devices
- Physical testing device: Gen 7 2019 iPad ~ \$100 on Ebay

Common Tools



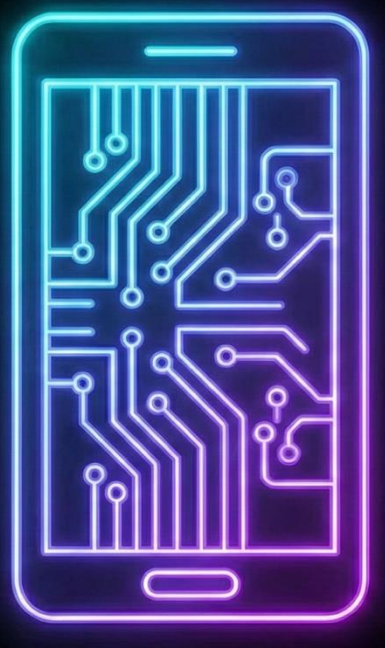
- . Frida
- . MobSF
- . Objection
- . Mariana Trench
- . JADX and JADX-GUI

Frida



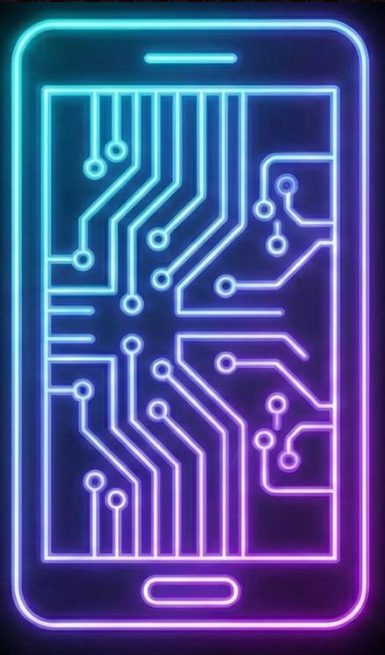
- Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers
- Injects a JavaScript engine into a local/remote process
- Interactive REPL and scriptable
- Portable – Most common architectures supported

MobSF



- . A static and dynamic analysis tool for Android and iOS mobile apps
- . Most often used for static analysis / recon first steps
- . Demo...

Mariana Trench



- A security focused static analysis platform targeting Android.

 Issue 2 Traces

Code 1

Description User input flows into code execution sink (RCE): Values from user-controlled source may eventually flow into code execution sink

Status 🚩 likely new

Callable void MainActivity.onCreate(Bundle)

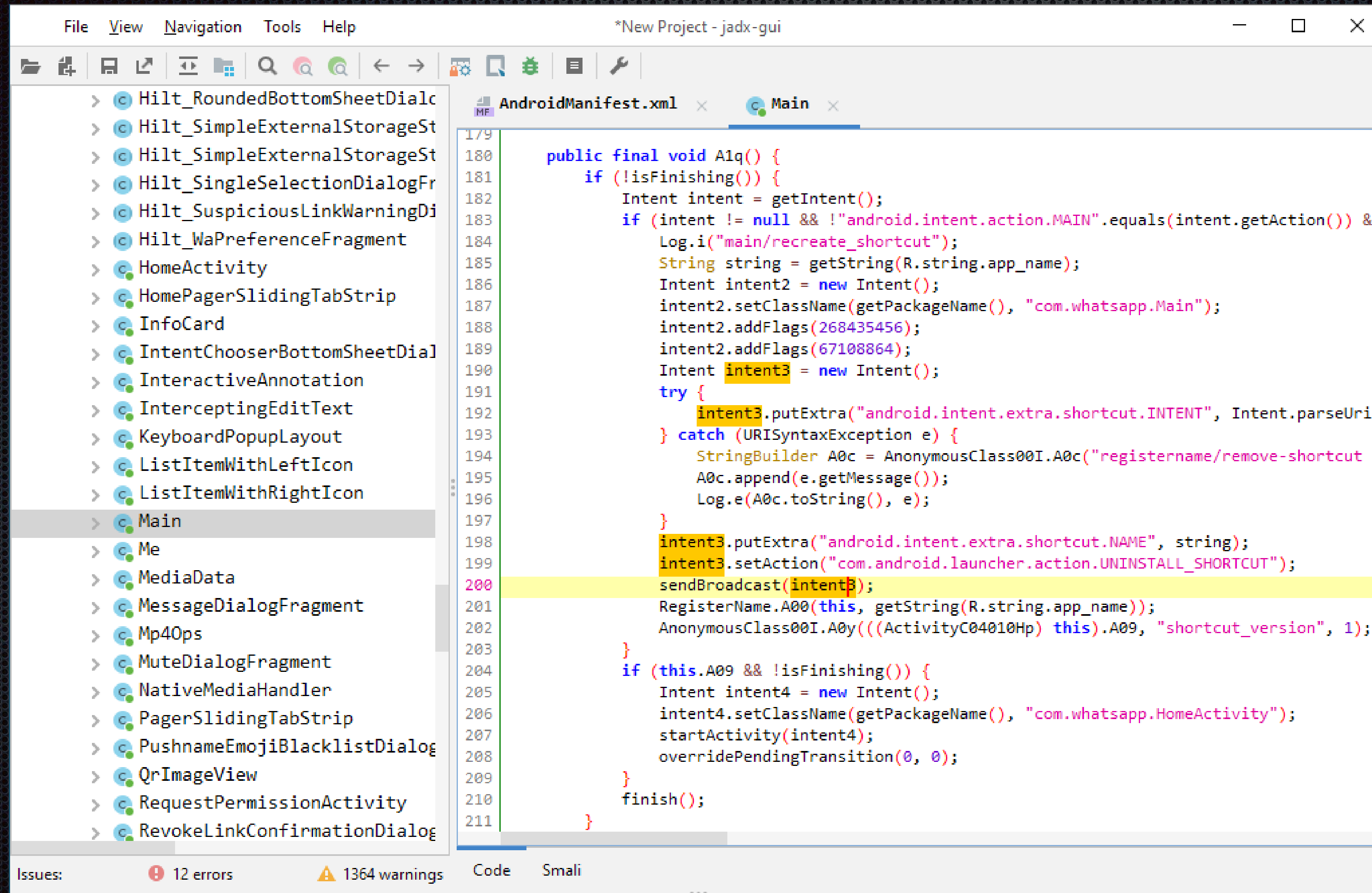
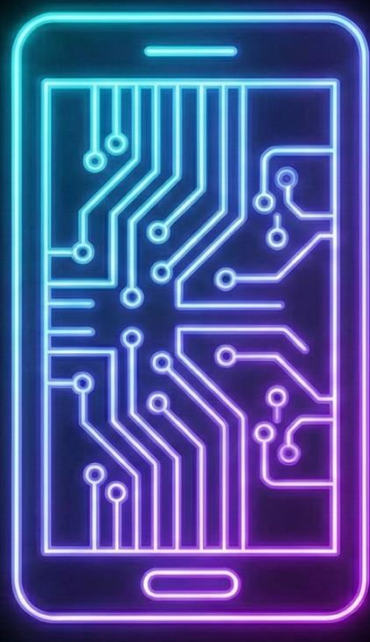
Location com/example/myapplication/MainActivity.java

Sources ActivityUserInput at minimum distance 1.
Landroid/app/Activity;.getIntent():()Landroid/content/Intent;

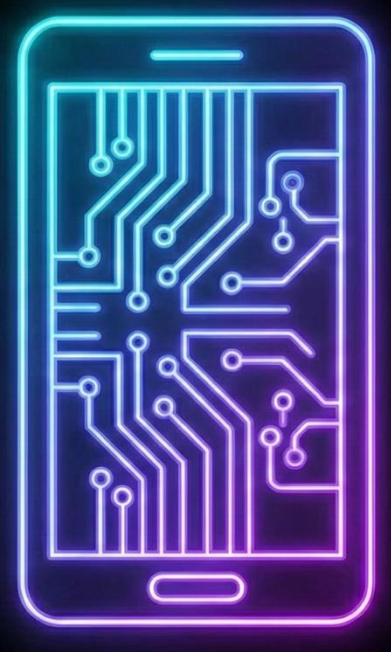
Sinks CodeExecution at minimum distance 3.
Ljava/lang/ProcessBuilder;<init>:([Ljava/lang/String;)V

Features always-via-anonymous-class-to-obscure always-via-array always-via-caller-exported always-via-obscure always-via-obscure-taint-in-taint-out
⊕ Show 5 more...

JADX-GUI

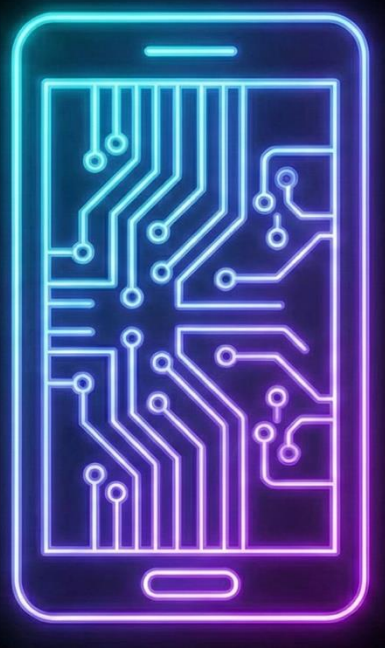


Intro to the OWASP MASTG



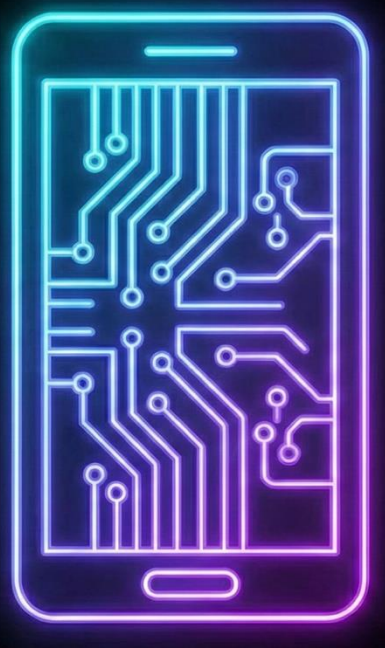
- Global standard for mobile app testing
- Knowledge | Techniques | Tools | Best Practices | Tests | Demos | Apps
- Risk-based tiering: L1 and L2
- Full surface coverage
- Can be overwhelming and very difficult to fit into a time-boxed pentest

OWASP MASTG Demo



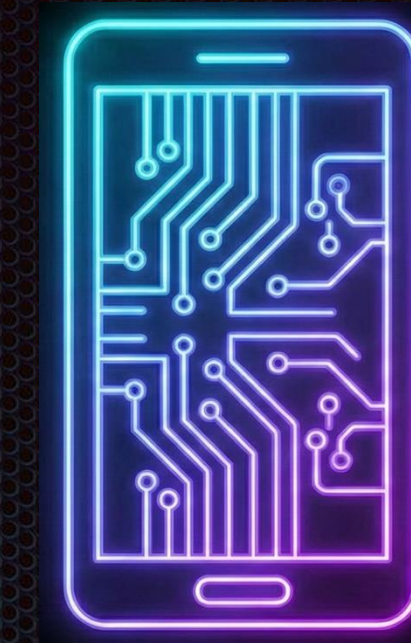
- Browse the site
- Examine the layers
- Explore the benefits

Fitting the MASTG into the time box



- Automation with Frida scripts and shell functions
- Frida CodeShare: -c, --codeshare CODESHARE_URI
- Trellis (tool drop)

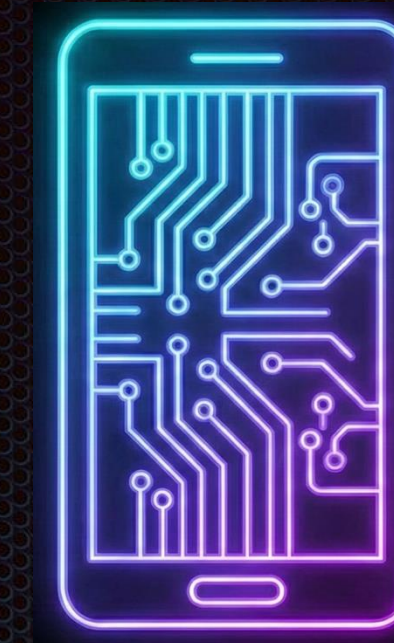
Example MASTG Automation (1)



List all non-Apple applications:

```
function csec-ios-list-apps() {  
    local SEARCH_TERM="$1"  
    ipsw idev apps ls | awk '{print $1, $2, $3, $4}' | grep -i "$SEARCH_TERM"  
}
```

Example MASTG Automation (2)



Set app package name (com.example.app) to variable:

```
function csec-ios-apn-to-variable() {
    local APN="$1"
    # 1. Run the command and capture the output into a variable
    search_result=$(ipsw idev apps ls | awk '{print $1, $2, $3, $4}' | grep -i "$APN")
    # 2. Count the number of non-empty lines in the result
    match_count=$(echo "$search_result" | grep -c .)
    # 3. If (and only if) there is exactly 1 match, extract the text inside ()
    if [ "$match_count" -eq 1 ]; then
        # -F'[''()']' tells awk to split the line by '(' and ')'
        # print $2 prints the text inside the first pair of parentheses
        APP_ID=$(echo "$search_result" | awk -F'[''()']' '{print $2}')
        APP_PATH=$(echo "$search_result" | awk '{print $4}')
    fi
}
```

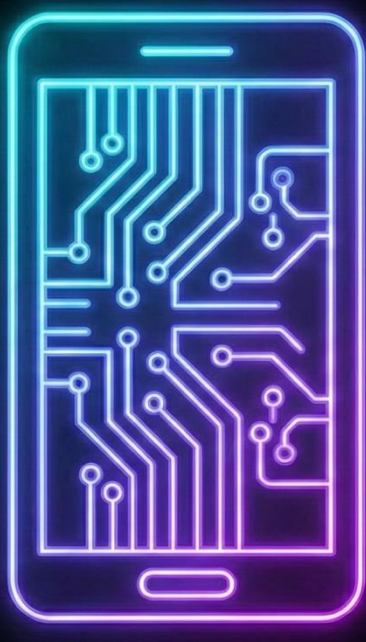


<https://codeshare.frida.re/@sdcampbell/dump-ios-encryption-ciphers/>

Key (128 bits):

Key Reused: **true**

IV Reused: true



Example finding: Cached Card Data

<https://codeshare.frida.re/@sdcampbell/check-ios-keyboard-cache/>

```
'<MDCTextField: 0x10729c600; baseClass = UITextField; frame = (0 0; 271  
or = UIExtendedGrayColorSpace 0.5 1; gestureRecognizers = <NSArray: 0x281  
TextFieldNoBackgroundProvider: 0x28189a2e0: textfield=<MDCTextField 0x107
```

```
'Field Name: Placeholder: *Card Number'} data: None
```

```
'is Editable: true'} data: None
```

```
'secureTextEntry: false'} data: None
```

```
'autocorrectionType: 0 (UITextAutocorrectionTypeDefault)'} data: None
```

```
'<MDCTextField: 0x10729ec00; baseClass = UITextField; frame = (224 0; 10  
olor = UIExtendedGrayColorSpace 0.5 1; gestureRecognizers = <NSArray: 0x2  
eldNoBackgroundProvider: 0x28189ac70: textfield=<MDCTextField 0x10729ec00
```

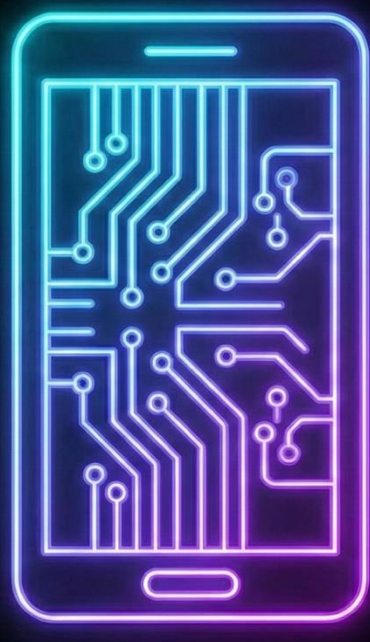
```
'Field Name: Placeholder: *CVV'} data: None
```

```
'is Editable: true'} data: None
```

```
'secureTextEntry: false'} data: None
```

```
'autocorrectionType: 0 (UITextAutocorrectionTypeDefault)'} data: None
```

Example finding: Insecure Keychain Storage

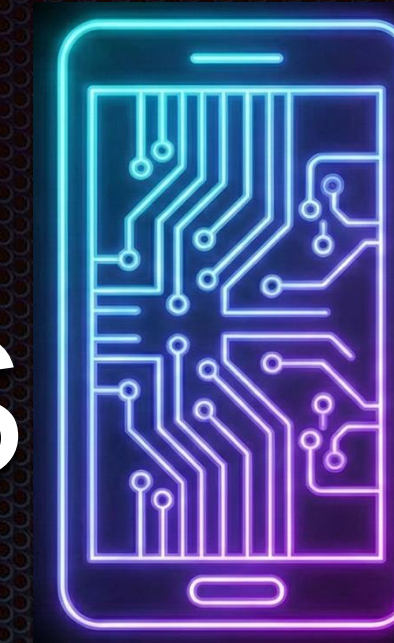


Type	Account	Service	Data
Password	1: [REDACTED]	FIRAPP_DEFAULT	
Password	_pro		2
Password	PWD		X [REDACTED] X
Password	accountNumber		X [REDACTED] X
Password	accountPassword		X [REDACTED] X
Password	rememberState		0

The default (vulnerable) keychain setting is `kSecAttrAccessible:kSecAttrAccessibleWhenUnlocked`, which leaves the data unprotected when a device passcode is not enforced.

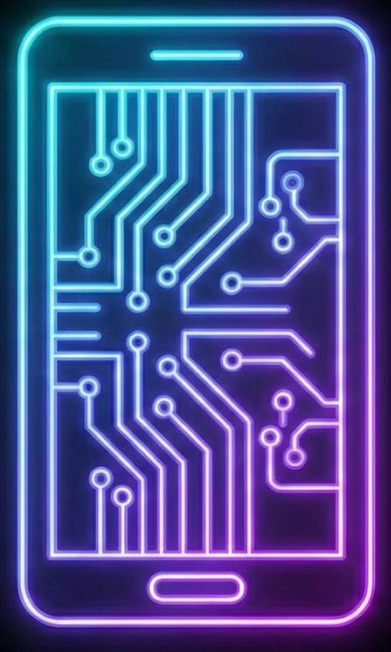
The recommended setting is `kSecAttrAccessible:kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly`. Using this setting, the app won't store credentials unless the user has set a passcode on the device.

Trellis: iOS Static -> Dynamic Analysis



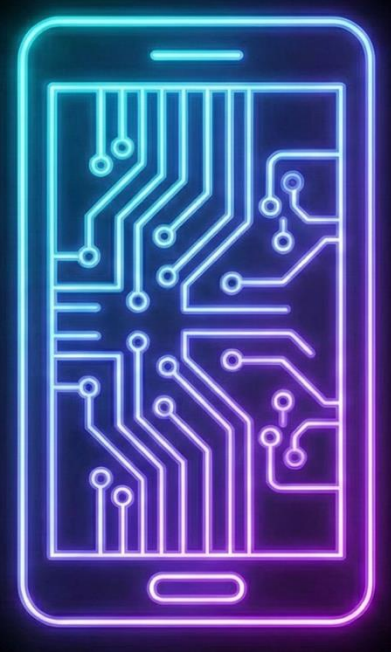
- Ghidra script
- Static analysis of iOS binaries - detects security-relevant function usage
- Frida script generation based on static analysis
- Goal: To provide a good balance between eliminating false-positives and false-negatives
- Memory addresses and offsets are provided in report findings to aid the analyst's investigation

Enhanced Detection: ARM64 Backward Slice



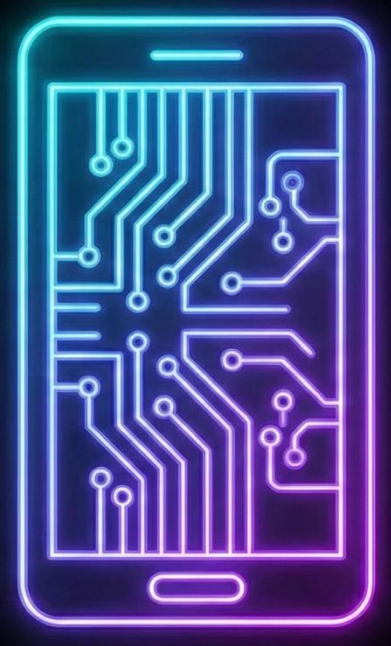
- **The Problem:** Ghidra's decompiler often can't resolve parameter values for Swift-to-C bridged calls
- Examples: CCCrypt, CCKeyDerivationPBKDF, CCHmac
- Without parameter values, we can't determine the algorithm, key size, or iteration count
- **The Solution:** Scan ARM64 instructions backward from the call site
- Look for MOV/MOVZ/ADRP+ADD patterns that load values into argument registers (x0-x7)

Enhanced Detection: Caller-Name Heuristics



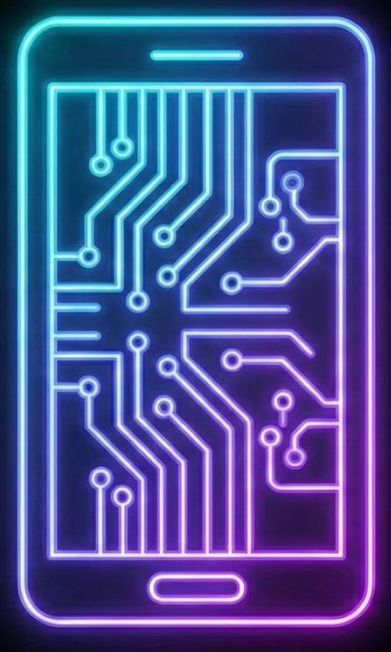
- **The Problem** - Even with backward slicing, some parameters remain unresolved
- We still need to flag these calls as security-relevant for the analyst
- **The Solution:** Inspect the calling function's name for security-relevant keywords
- Applied across multiple security domains: crypto, jailbreak, anti-debug
- **Example:** A caller named "BrokenCrypto...pbkdf2" calling CCEncrypt = crypto review needed

Enhanced Detection: ObjC Selector Matching



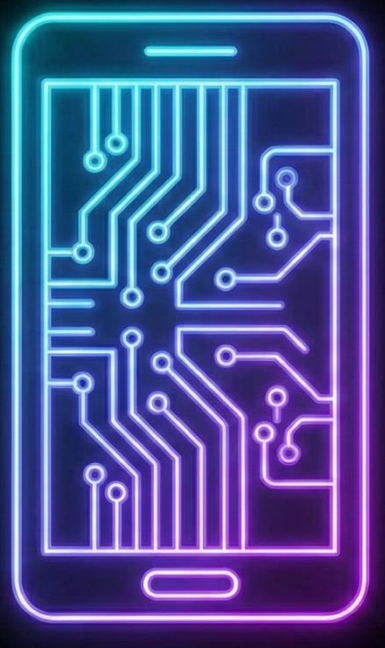
- **The Problem:** ObjC methods are not standalone functions in the binary
- They are dispatched via `objc_msgSend` with the selector as a string argument
- Invisible to traditional import scanning
- **The Solution:** Search the string table for selector strings matching our signatures
- Follow xrefs from matched strings to locate the actual call sites

Enhanced Detection: String Cross-Referencing



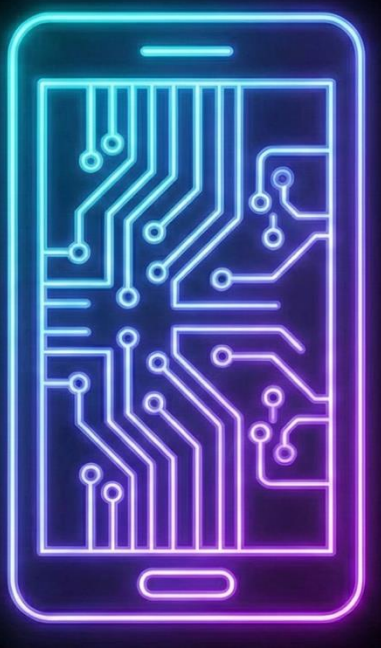
- **The Problem:** String scanning finds hardcoded passwords and API keys
- But without context: is a suspicious string used in crypto, auth, or just a UI label?
- **The Solution:** Use Ghidra xrefs to determine WHICH FUNCTIONS reference each suspicious string
- Links hardcoded passwords to crypto call sites
- Links HTTP URLs to auth/payment functions
- Dramatically reduces false positives

Trellis demo



- Review Markdown reports
- Examine generated Frida scripts

Finale



- Questions or Comments?
- Contact:
 - X: @cylentsec
 - Email: scampbell@cylentsec.com
 - Web: www.cylentsec.com, github.com/cyclentsec